

The background is a vibrant blue gradient with a complex, isometric digital architecture. It features several 3D rectangular blocks of varying sizes, some of which are hollow or have glowing blue interiors. These blocks are interconnected by a network of lines and dots, suggesting a data flow or a complex system. Scattered throughout the scene are numerous small, glowing dots in red, green, and blue, along with thin, bright lines that create a sense of depth and movement. The overall aesthetic is futuristic and technological.

Arquitecturas Avanzadas de Software

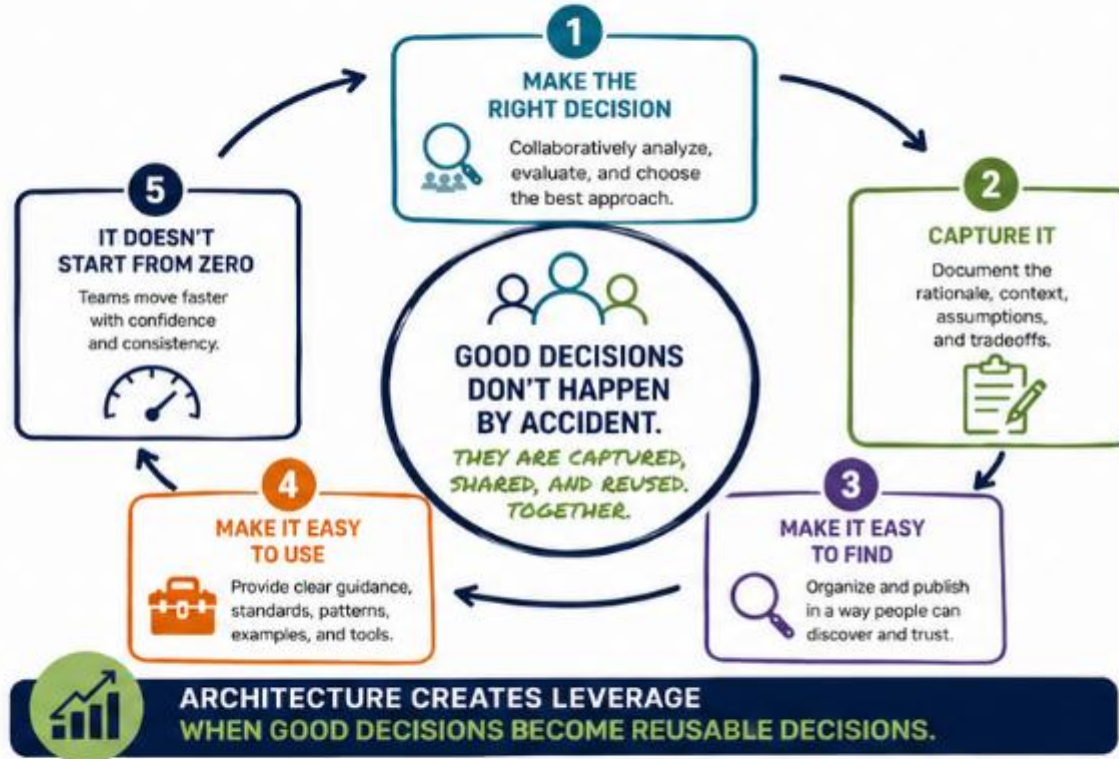
Universidad EAFIT
Septiembre 2026

REVISEMOS

el ensayo!!

THE BEST ARCHITECTURE DECISION IS ONE YOU DON'T HAVE TO MAKE TWICE

Good decisions. Captured together. Used by everyone. Creating leverage for all.



HOW IT MAKES IT BETTER FOR IT AND THE BUSINESS



FASTER DELIVERY

Teams spend less time deciding and more time delivering business value.



LOWER RISK

Proven approaches reduce uncertainty, defects, and security gaps.



REDUCE COST

Reuse and standards eliminate duplication and unnecessary complexity.



GREATER ALIGNMENT

Common patterns and standards create consistency across teams.



IMPROVED QUALITY

Built-in architectural guardrails lead to reliable, scalable, and maintainable solutions.



STRONGER INNOVATION

Freed-up capacity allows teams to focus on new ideas that matter.

“

Architecture doesn't work in isolation. We partner with IT to capture, refine, and deliver guidance that teams can trust and reuse.

TOGETHER, WE ENABLE BETTER IT OUTCOMES.



WHAT WE PROVIDE



STANDARDS

Clear rules that ensure consistency, security, and quality.



PATTERNS

Proven approaches to solve common problems in repeatable ways.



REFERENCE ARCHITECTURES

End-to-end views that show how everything works together.



GUIDES & PLAYBOOKS

Practical guidance for when, how, and why to use them.

“

OUR GOAL IS NOT TO CONTROL DECISIONS.
OUR GOAL IS TO EMPOWER BETTER ONES—TOGETHER.



Los atributos de calidad de una solución son propiedades medibles que determinan **qué tan bien un sistema** satisface las necesidades de los usuarios y los objetivos del negocio.

¿Por qué son importantes?

- ✓ Permiten **evaluar y comparar** soluciones
- ✓ Guían **decisiones** de diseño y tecnología
- ✓ Aseguran **valor, sostenibilidad** y alineación con los objetivos



Atributos clave



Seguridad

Protege datos
y accesos



Rendimiento

Responde rápido
y eficientemente



Escalabilidad

Crece según la
demanda



Disponibilidad

Está operativo
cuando se necesita



Modificabilidad

Se adapta a
nuevos cambios



Confiabilidad

Funciona de forma
correcta y consistente



Usabilidad

Fácil de usar y
comprender



Mantenibilidad

Fácil de mantener
y evolucionar



CARACTERÍSTICAS ARQUITECTÓNICAS

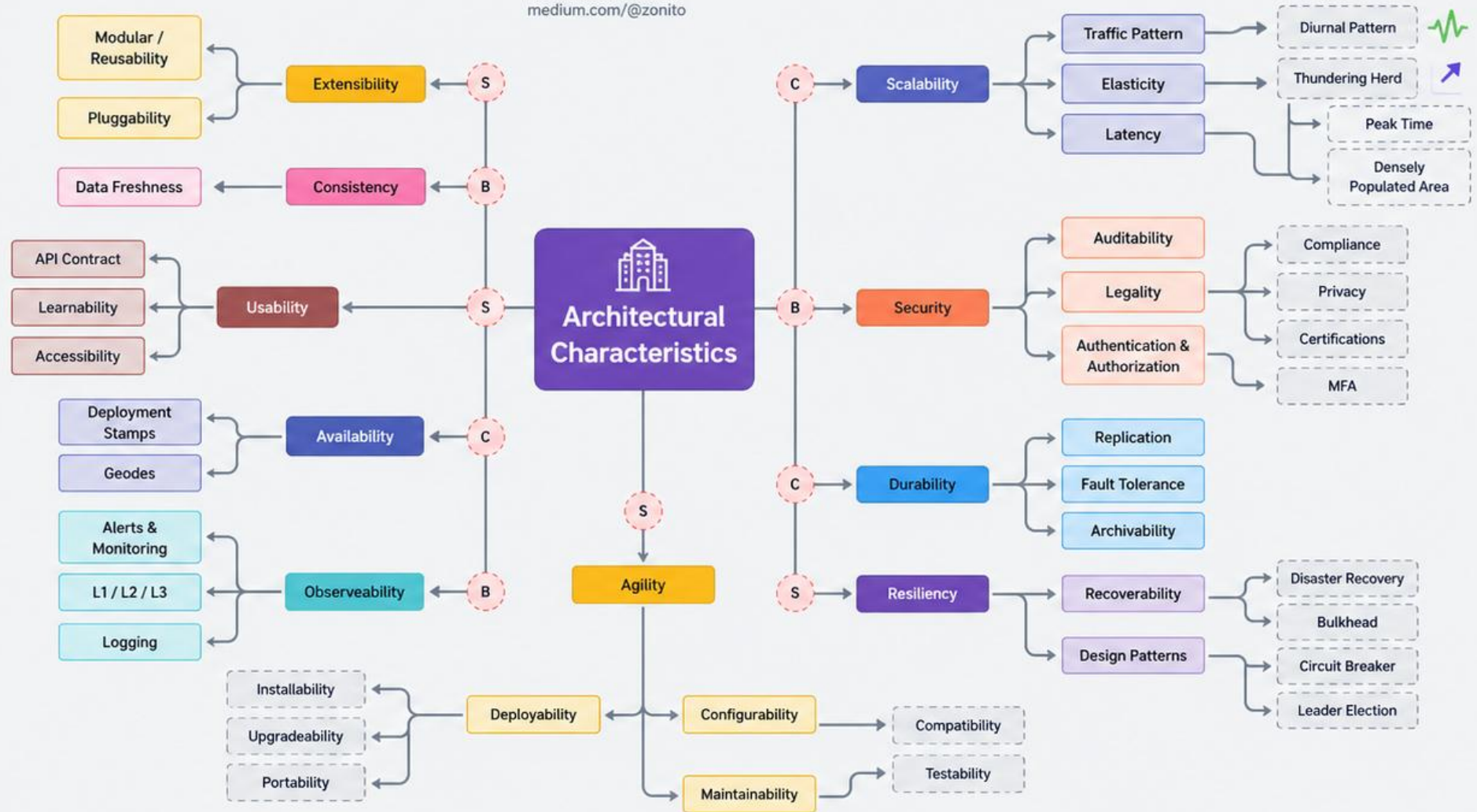
Principios y atributos que guían el diseño de soluciones robustas, escalables, seguras y sostenibles en el tiempo.

-  Diseño modular y adaptable
-  Seguridad y cumplimiento por diseño
-  Alta disponibilidad y resiliencia
-  Observabilidad y mejora continua

ARCHITECTURAL CHARACTERISTICS

Creational Structural Behavioral

medium.com/@zonito



Las características arquitectónicas son los pilares que aseguran que nuestras soluciones evolucionen con el negocio y entreguen valor de forma confiable, segura y eficiente.

ARCHITECTURE CHARACTERISTICS WORKSHEET



Estas características guían el diseño de soluciones robustas, escalables, seguras y sostenibles, asegurando valor real y eficiencia en el tiempo.



PERFORMANCE

El tiempo que tarda el sistema en procesar una solicitud de negocio.



RESPONSIVENESS

El tiempo que tarda el sistema en generar una respuesta al usuario.



AVAILABILITY

El tiempo de actividad del sistema; normalmente medido en 9's (ej., 99.9%).



FAULT TOLERANCE

Cuando ocurren errores fatales, otras partes del sistema continúan funcionando.



SCALABILITY

Función de la capacidad y el crecimiento del sistema; a medida que aumentan usuarios, solicitudes, capacidad, rendimiento y tasas de error se mantienen constantes.



ELASTICITY

El sistema puede expandirse y responder rápidamente a cargas extremas inesperadas o anticipadas (ej., de 20 a 250,000 usuarios instantáneamente).



DATA INTEGRITY

Los datos en todo el sistema son correctos y no hay pérdida de información.



DATA CONSISTENCY

Los datos en todo el sistema están sincronizados y son consistentes entre bases de datos y tablas.



ADAPTABILITY

Facilidad con la que el sistema se adapta a cambios en el entorno y la funcionalidad.



CONCURRENCY

Capacidad del sistema para procesar solicitudes simultáneas, en la mayoría de los casos en el mismo orden en que fueron recibidas; implícito cuando se soportan escalabilidad y elasticidad.



INTEROPERABILITY

Capacidad del sistema para interactuar con otros sistemas para completar una solicitud de negocio.



EXTENSIBILITY

Facilidad con la que el sistema puede extenderse con funciones adicionales.



DEPLOYABILITY

Tiempo y esfuerzo involucrado en liberar el software, la frecuencia de liberaciones y el riesgo general de despliegue.



TESTABILITY

Facilidad y completitud de las pruebas.



ABSTRACTION

Nivel en el que partes del sistema están aisladas de otras partes del sistema (tanto interacciones internas como externas).



WORKFLOW

Capacidad del sistema para gestionar flujos de trabajo complejos que requieren múltiples componentes (servicios) para completar una solicitud de negocio.

C

Creational

S

Structural

B

Behavioral



Las características arquitectónicas aseguran que nuestras soluciones evolucionen con el negocio, entregando valor de forma confiable, segura, eficiente y sostenible en el tiempo.



ARCHITECTURE CHARACTERISTICS WORKSHEET

Pilares que guían el diseño de soluciones robustas, adaptables y sostenibles en el tiempo.

C Creational
S Structural
B Behavioral



CONFIGURABILITY

Capacidad del sistema para soportar múltiples configuraciones, así como configuraciones personalizadas y actualizaciones de configuración bajo demanda.

C



RECOVERABILITY

Capacidad del sistema para arrancar desde donde se quedó en caso de una caída del sistema.

B



FEASIBILITY (IMPLICIT)

Consideración de marcos de tiempo, presupuestos y habilidades del desarrollador al tomar decisiones arquitectónicas; los marcos de tiempo ajustados y los presupuestos impulsan esta característica arquitectónica.

S



SECURITY (IMPLICIT)

Capacidad del sistema para restringir el acceso a información sensible o funcionalidad.

B



MAINTAINABILITY (IMPLICIT)

Nivel de esfuerzo requerido para localizar y aplicar cambios en el sistema.

B



OBSERVABILITY (IMPLICIT)

Capacidad de un sistema o servicio para hacer disponibles y transmitir métricas como salud general, tiempo de actividad, tiempos de respuesta, rendimiento, etc.

B



CONFIGURABILITY (IMPLICIT)

Capacidad del sistema para soportar múltiples configuraciones, así como configuraciones personalizadas y actualizaciones de configuración bajo demanda.

C



RECOVERABILITY (IMPLICIT)

Capacidad del sistema para arrancar desde donde se quedó en caso de una caída del sistema.

B



FEASIBILITY (IMPLICIT)

Consideración de marcos de tiempo, presupuestos y habilidades del desarrollador al tomar decisiones arquitectónicas; los marcos de tiempo ajustados y los presupuestos impulsan esta característica arquitectónica.

S



SECURITY (IMPLICIT)

Capacidad del sistema para restringir el acceso a información sensible o funcionalidad.

B



MAINTAINABILITY (IMPLICIT)

Nivel de esfuerzo requerido para localizar y aplicar cambios en el sistema.

B



OBSERVABILITY (IMPLICIT)

Capacidad de un sistema o servicio para hacer disponibles y transmitir métricas como salud general, tiempo de actividad, tiempos de respuesta, rendimiento, etc.

B



Estas características arquitectónicas aseguran que nuestras soluciones evolucionen con el negocio, entregando valor de forma confiable, segura y eficiente en el tiempo.

Características de una Arquitectura

Cómo las decisiones arquitectónicas generan valor para el negocio

Las características de la arquitectura determinan cómo un sistema satisface sus requisitos de calidad y genera valor sostenible para los interesados.

Performance

Capacidad de responder rápido y procesar grandes volúmenes de trabajo.



Escalabilidad

Capacidad de crecer y adaptarse a la demanda sin degradar el servicio.



Mantenibilidad

Facilidad para entender, modificar y evolucionar el sistema.



Arquitectura del Sistema

Seguridad

Protección de datos, sistemas y usuarios ante amenazas.



Disponibilidad / Resiliencia

Capacidad de operar de forma continua y recuperarse ante fallos.



Interoperabilidad

Facilidad para integrarse con otros sistemas y tecnologías.



“

Las características son las propiedades más importantes de un sistema desde la perspectiva de los interesados.”.

— Mark Richards,
Software Architecture Patterns



Una buena arquitectura optimiza los trade-offs para maximizar el valor entregado al negocio y a los usuarios.

De las Características Arquitectónicas al Valor de Negocio

Satisfacción de Usuarios

- Tiempo de respuesta
- Disponibilidad
- Estabilidad
- Experiencia consistente

Cientes más satisfechos y mayor retención.



Time to Market

- Despliegues más rápidos
- Menos retrabajo
- Releases pequeños
- Menor riesgo

Más funcionalidades en menor tiempo.

Mergers & Acquisitions

- Integrar nuevas empresas
- Consumir APIs externas
- Incorporar nuevos canales
- Migrar plataformas

Integraciones en semanas en vez de años.

Ventaja Competitiva

- Innovación continua
- Escalabilidad
- Adaptación al mercado
- Resiliencia

La empresa cambia más rápido que sus competidores.



Arquitectura
(Sustenta el sistema)



Calidad Técnica
(Garantiza requisitos)



Valor para el Negocio
(Genera resultados)



Diferenciación Competitiva
(Crea ventaja sostenible)

ARQUITECTURAS
AVANZADAS DE SOFTWARE

Arquitecturas Evolutivas de Software

Enfoques arquitectónicos que permiten que los sistemas **evolucionen de manera incremental y continua** para adaptarse a nuevas necesidades, tecnologías y contextos de negocio, sin comprometer su estabilidad ni calidad.



ADAPTABILIDAD

Responde al cambio de forma continua.



FLEXIBILIDAD

Permite extender y reconfigurar fácilmente.



RESILIENCIA

Mantiene la estabilidad ante la evolución.



VALOR CONTINUO

Entrega iterativa de valor al negocio.



DEFINICIÓN TÉCNICA

Las arquitecturas evolutivas son enfoques de diseño que priorizan la **modularidad**, la **desacoplación** y la **iteración continua** para facilitar el cambio controlado y sostenible del sistema a lo largo del tiempo.





¿Qué es una Arquitectura Evolutiva?



"Una arquitectura evolutiva soporta el cambio guiado e incremental **como primer principio**, a través de múltiples dimensiones."

— Ford, Parsons & Kua · Building Evolutionary Architectures (O'Reilly, 2017)



Cambio Incremental

El sistema **se adapta gradualmente** sin necesidad de rediseños masivos ni "big bang" migrations.



Múltiples Dimensiones

Considera: **rendimiento, seguridad, escalabilidad, mantenibilidad, legalidad y experiencia de usuario.**



Cambio Guiado

Las **fitness functions** actúan como guardianes automatizados que validan cada cambio arquitectónico.



Arquitecturas Ágiles vs. Arquitecturas Evolutivas

Dos enfoques complementarios para manejar el cambio en el software



Arquitectura Ágil

VS



Arquitectura Evolutiva

 Enfoque	Responder al cambio de requerimientos con ciclos cortos (sprints)	»»	Guiar el cambio arquitectónico mediante métricas y fitness functions
 Origen	Manifiesto Ágil (2001) Prácticas de desarrollo de software	»»	Evolución del diseño técnico Basado en teoría de sistemas complejos
 Métricas	Velocidad de entrega, cobertura de pruebas, deuda técnica	»»	Fitness functions: umbrales medibles por dimensión arquitectónica
 Cambio	Iterativo centrado en funcionalidades (user stories, backlog)	»»	Incremental centrado en la estructura del sistema a lo largo del tiempo
 Herramientas	Scrum, Kanban, CI/CD , TDD, BDD	»»	Fitness functions automatizadas, arquitectural fitness pipelines



La arquitectura evolutiva **NO reemplaza** a la ágil — la **complementa** agregando una capa de validación estructural continua.



Fitness Functions — Definición



Una **Fitness Function** es un mecanismo de evaluación objetivo que **proporciona una medida de qué tan cerca** está un artefacto de software de alcanzar un objetivo arquitectónico deseado.

¿Para qué sirven?



Proteger **características arquitectónicas** críticas ante el cambio.



Verificar **automáticamente** que la arquitectura sigue siendo válida con cada deploy.



Formalizar **restricciones** (ej: latencia < 200ms, cobertura > 80%).



Crear un pipeline de **validación arquitectónica continua**.

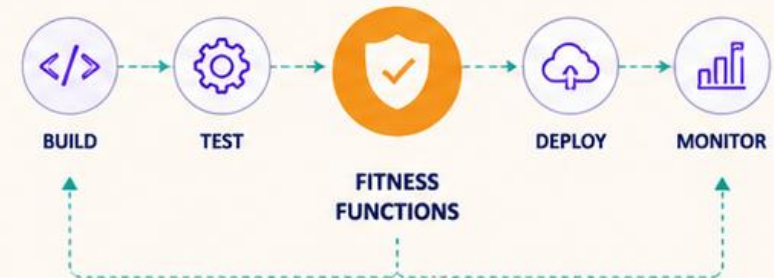


Analogía

Como los **tests unitarios** validan el comportamiento del código, las **fitness functions** validan la integridad de la arquitectura.

Un pipeline de **CI/CD** puede fallar si:

- La latencia supera **200ms**
- El **acoplamiento** entre módulos aumenta
- La **cobertura** cae por debajo del umbral
- Un servicio viola el **SLA de disponibilidad**

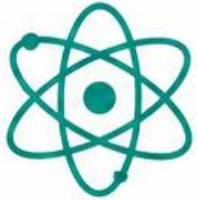


Taxonomía de Fitness Functions



Las fitness functions se clasifican en dos dimensiones ortogonales:

ATÓMICA



Evalúa **UNA** sola característica arquitectónica en **aislamiento**.

- Ej: cobertura de pruebas de un módulo específico.

HOLÍSTICA

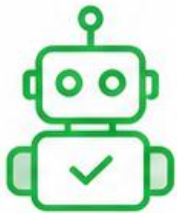


Evalúa **múltiples características** en **combinación**.

- Ej: rendimiento + disponibilidad + seguridad simultáneamente.



AUTOMATIZADA



Ejecutada por **herramientas sin intervención humana**.

- Ej: test en pipeline CI/CD que verifica latencia de API.

MANUAL



Requiere **criterio humano** para su evaluación.

- Ej: revisión de auditor sobre cumplimiento RGPD/SOX.



Fitness Functions: **Automatizada** vs. **Manual**



Dos enfoques complementarios para evaluar la arquitectura de software



AUTOMATIZADA

Ejecutada sin intervención humana

Úsala para: rendimiento, disponibilidad, acoplamiento, cobertura, seguridad técnica

Herramientas / Mecanismos:

-  JUnit / TestNG (pruebas)
-  ArchUnit (restricciones arch)
-  Gatling / k6 (rendimiento)
-  SonarQube (calidad código)
-  OWASP ZAP (seguridad)
-  Prometheus + Grafana (métricas)

- ✓ Ejecución continua en CI/CD
- ✓ Resultados reproducibles

- ⚠ No detecta problemas semánticos
- ⚠ Requiere inversión inicial en setup



MANUAL

Requiere criterio y juicio humano

Úsala para: cumplimiento legal, revisiones de seguridad, experiencia de usuario, ethics AI

Herramientas / Mecanismos:

-  Revisiones de arquitectura (ADR)
-  Auditorías externas
-  Pruebas de usabilidad (A/B)
-  Análisis de riesgo regulatorio
-  Revisión de privacidad (DPO)
-  Threat modeling (STRIDE)

- ✓ Captura contexto y semántica
- ✓ Obligatoria para cumplimiento regulatorio

- ⚠ Costosa y no escala fácilmente
- ⚠ Sujeta a sesgos humanos



Caso Real: DHL — **Arquitectura Evolutiva** en Logística Global



DHL Express — Líder mundial en logística con operaciones en **220+** países, **600,000+** empleados y **1.8 billones** de envíos/año

El Desafío Arquitectónico

-  Monolito de **15+ años** con más de **40 millones** de líneas de código
-  **98 sistemas** legados interdependientes de misión crítica
-  **3.000+ integraciones** con partners, aduanas y aerolíneas
-  Necesidad de **escalar independientemente** por región y servicio
-  SLAs contractuales con clientes premium: **disponibilidad 99.99%**



Decisión Arquitectónica

Migración **evolutiva** (no big-bang) hacia **microservicios** sobre plataforma cloud-native usando:

- ✓ Strangler Fig Pattern para desacoplar el monolito gradualmente
- ✓ Fitness functions por cada bounded context migrado
- ✓ Architecture Decision Records (ADR) como trazabilidad
- ✓ Observabilidad con **Dynatrace + Prometheus**



220+
Países



1,800
Vuelos/día



40M+
Líneas código



99.99%
SLA target



Caso Real: DHL — Fitness Functions Implementadas



FF-01



Disponibilidad por Ruta

Atómica · Automatizada

El servicio de rastreo de envíos debe tener **uptime $\geq 99.95\%$** medido con synthetic monitoring cada 60 segundos.

 **$\geq 99.95\%$ mensual**

Dynatrace Synthetic + PagerDuty

FF-02



Latencia End-to-End de Reserva

Holística · Automatizada

El flujo completo de creación de envío (cotización→reserva→confirmación) debe completarse en **$< 2.5s$** en percentil p95 bajo carga normal.

 **< 2.5 segundos (p95)**

Gatling + Grafana Dashboard

FF-03



Cumplimiento Aduanero

Atómica · Manual

100% de los envíos internacionales deben incluir documentación aduanera válida según la regulación del país destino. Verificado trimestralmente por equipo legal.

**100% (cero tolerancia)**Revisión trimestral equipo Legal
+ Customs API validation

FF-04



Resiliencia ante Falla de Nodo

Holística · Automatizada

El sistema de ruteo de paquetes debe mantener operación completa ante la falla simultánea de **hasta 2 nodos** en cualquier región.

**RTO < 30 segundos**
• RPO = 0Chaos Monkey (Netflix OSS)
+ AWS FIS

Resultado: DHL redujo incidentes en producción en un **67%** y el MTTR de **4h a 18 min** en los servicios migrados.



El reto: cambiar el motor con el avión en vuelo



¿Por qué no podemos simplemente **detener** el sistema y **rehacerlo**?



Riesgo Big Bang



Reescribir el sistema completo antes de ponerlo en producción multiplica el riesgo.

El **70%** de proyectos big bang fracasan o se entregan con retraso.



Continuidad operacional



Un retailer no puede bajar sus sistemas en Black Friday, Navidad ni fin de mes.

El negocio **no para**; la arquitectura tampoco puede hacerlo.



Deuda técnica acumulada



Los sistemas heredados no desaparecen de un día para otro.

Conviven con los nuevos durante meses o años; el patrón elegido determina qué tan caro es ese período de **coexistencia**.

“

*“El cambio incremental no es una opción de comodidad — es **la única estrategia viable** en sistemas con alta disponibilidad requerida.”*





PATRONES DE

CAMBIO INCREMENTAL

en Arquitectura de Software

EAFIT · Arquitecturas Avanzadas de SW

Cuatro estrategias para transformar sistemas sin detener el negocio



Strangler Fig



Feature Toggles



Branch by Abstraction



Parallel Run



Caso de aplicación: empresa de Retail





01 · STRANGLER FIG PATTERN

 Definición

Técnica de **migración** donde el sistema nuevo envuelve **progresivamente** al sistema heredado. Las funcionalidades se migran ruta a ruta hasta que el legacy queda “**estrangulado**” y puede retirarse sin un corte abrupto.



La metáfora: el árbol higuero estrangulador



Principio clave: el proxy decide qué peticiones va al legacy y cuáles al nuevo sistema. El rollback es trivial: basta re-enrutar.



Permite mejorar de forma **incremental**, reduciendo el **riesgo** y el **impacto** en el negocio.





Ejemplo Retail — FalabellaStore

Escenario: Módulo de checkout monolítico (Oracle ATG, 15 años de antigüedad) se migra a microservicios.

- 1 Se instala un API Gateway (Kong) frente al monolito.
- 2 El carrito de compras se extrae como microservicio React + Node. El Gateway enruta **/cart** al nuevo servicio.
- 3 Dos semanas: solo nuevos usuarios van al microservicio (canary 5% → 50% → 100%).
- 4 Se extrae **/checkout/payment**, luego **/checkout/shipping**.
- 5 En 9 meses, el **80%** del flujo pasa por microservicios. El monolito atiende solo fulfillment legacy.
- 6 En mes 14 se retira el monolito. **Cero downtime** registrado.



Cuándo NO aplicar

- ✗ El sistema legacy no tiene interfaz HTTP bien definida (p.ej. batch de noche sin API).
- ✗ El dominio tiene alta cohesión interna: extraer una parte rompe invariantes de negocio.
- ✗ Equipo < 5 personas sin experiencia en API Gateways ni observabilidad distribuida.
- ✗ El plazo es < 3 meses: el patrón requiere tiempo de coexistencia.



Criterios para aplicarlo

- ✓ Sistema legacy con APIs o URIs identificables para enrutar.
- ✓ Capacidad de desplegar un proxy / API Gateway en producción.
- ✓ Al menos una funcionalidad con bajo acoplamiento para empezar.
- ✓ Equipo dispuesto a mantener el legacy MIENTRAS migra.
- ✓ Presupuesto para coexistencia de infraestructura 6-18 meses.





02 • FEATURE TOGGLES



Mecanismo que permite **activar o desactivar funcionalidades** en producción **SIN desplegar** nuevo código, controlando la exposición a **segmentos de usuarios, regiones o contextos específicos**.

Tipos de Feature Toggles



Release Toggle

Ocultar funciones en desarrollo hasta estar listas



Vida útil:
Días / semanas



Experiment Toggle

A/B testing: versión A vs B para medir conversión



Vida útil:
Semanas



Ops Toggle

Circuit breaker: deshabilita módulos bajo alta carga



Vida útil:
Horas / permanente



Permission Toggle

Activación por segmento: usuarios premium, país, rol



Vida útil:
Permanente





Ejemplo Retail — Éxito.com

Escenario: Lanzamiento del nuevo motor de recomendaciones ML en plataforma de e-commerce con **3,2 millones** de usuarios activos.



Release Toggle

El nuevo motor ML se despliega “apagado”.
Se hace QA en producción con empleados internos.



Permission Toggle

Se activa para usuarios con tarjeta Éxito (segmento premium). **15%** de la base.



Experiment Toggle

A/B test: **50%** premium ve ML, 50% ve el motor anterior.
Se mide CTR y conversión 2 semanas.



Ops Toggle

Durante el Cyber Monday, si la latencia del modelo ML supera **300ms**, el toggle la apaga automáticamente y sirve recomendaciones estáticas.



Release Toggle → OFF

Tras validar **+12%** conversión, el toggle se elimina del código. Motor ML es el default.



Cuándo NO aplicar

- ✗ Para cambios estructurales de BD (schema migrations) — el toggle no controla el estado de datos.
- ✗ Cuando los toggles se acumulan sin fecha de expiración: ‘toggle debt’ degrada la mantenibilidad.
- ✗ Funcionalidades con dependencias circulares entre toggles: A activo requiere B activo.
- ✗ Cuando no hay sistema de gestión centralizado (LaunchDarkly, Split.io, Unleash): toggles dispersos en código son inmanejables.



Criterios para aplicarlo

- ✓ Despliegues frecuentes (CI/CD) con necesidad de separar deploy de release.
- ✓ Segmentación de usuarios disponible (ID de usuario, región, plan).
- ✓ Necesidad de rollback instantáneo sin redesplicue.
- ✓ Infraestructura de gestión de toggles disponible o presupuestada.
- ✓ El equipo tiene disciplina para eliminar toggles vencidos en cada sprint.





Técnica de refactoring donde se introduce una **capa de abstracción** (interfaz) sobre un componente a reemplazar. El componente nuevo y el viejo implementan la misma abstracción, permitiendo **conmutar entre ellos** sin tocar el código cliente.

Los 5 pasos del patrón

1

**Crear la abstracción:**

Definir interfaz o clase abstracta que representa el comportamiento actual.

2

**Hacer que el cliente use la abstracción:**

Refactorizar el código que llama al componente para que pase por la interfaz.

3

**Construir la implementación nueva:**

Desarrollar el nuevo componente que implementa la misma interfaz.

4

**Alternar implementaciones:**

Usar un feature toggle o inyección de dependencias para conmutar.

5

**Eliminar la implementación vieja:**

Una vez validado el nuevo componente, se retira el viejo y opcionalmente la abstracción.





Ejemplo Retail — Retail360 (WMS)

Escenario: Reemplazar el motor de cálculo de inventario disponible (ATP) de SAP legacy por un motor en tiempo real basado en eventos (Kafka + Redis).



Abstracción

Se crea interfaz **InventoryService** con métodos: `getAvailable(sku, warehouse)`, `reserve(sku, qty)`.



Refactoring

El módulo de pedidos pasa a llamar **InventoryService** en lugar de llamar directamente a SAP BAPI.



Impl. nueva

Se desarrolla **KafkaInventoryService** que implementa **InventoryService** consultando Redis con datos en tiempo real.



Conmutación

Feature toggle activo **KafkaInventoryService** para el almacén piloto (Itagüí). El resto sigue con **SAPInventoryService**.



Retiro

Validado el piloto en 3 almacenes, se migra el 100% y se elimina **SAPInventoryService** + abstracción.



Cuándo NO aplicar



El código heredado no permite inyección de dependencias (p.ej. llamadas estáticas o globales entrelazadas).



El equipo no domina los principios SOLID — la abstracción mal diseñada introduce más acoplamiento.



La implementación nueva requiere un cambio de modelo de datos incompatible: necesita migración de datos previa.



Plazo de coexistencia > 6 meses sin plan claro de retiro: aumenta 'implementation debt'.



Criterios para aplicarlo



El componente a reemplazar tiene una interfaz semánticamente bien definida.



El lenguaje/framework soporta DI (Spring, .NET Core, NestJS, etc.).



Un solo componente se reemplaza, no múltiples con dependencias cruzadas.



El equipo puede desarrollar la nueva implementación en paralelo al main trunk.



Existe una suite de pruebas (contrato) que valida que ambas implementaciones son equivalentes.



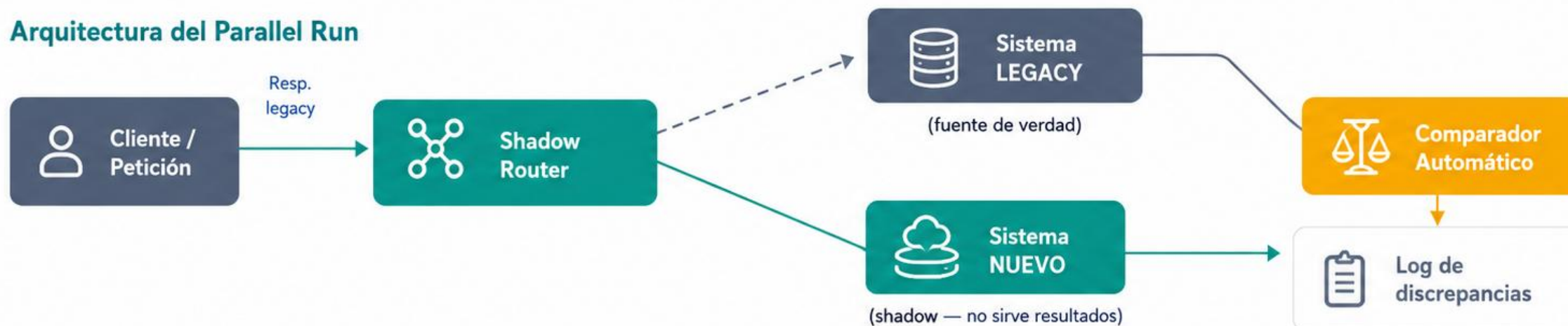


04 • PARALLEL RUN



Estrategia en la que el sistema nuevo y el sistema legado ejecutan simultáneamente el **MISMO conjunto de transacciones reales**. Los resultados se comparan automáticamente (*shadowing*). El sistema legado sigue siendo la fuente de verdad hasta que se valida la equivalencia.

Arquitectura del Parallel Run



El cliente **SIEMPRE** recibe la respuesta del sistema legacy. El sistema nuevo corre en modo 'sombra': procesa todo pero sus resultados solo van al log de comparación. La confianza crece con cada ciclo de reconciliación.





Ejemplo Retail — Grupo Cencosud

Escenario: Migración del motor de pricing dinámico (reglas de descuento por categoría, cliente, región) de un sistema COBOL de 25 años a un motor en Java/Spring con reglas en Drools.

Semanas 1-4

Ambos motores reciben cada transacción de venta. El COBOL determina el precio final. El motor Drools corre en sombra.

Semanas 5-8

El comparador detecta **847** discrepancias en **320.000** transacciones (**0,26%**). Se analizan: el **82%** eran errores en la migración de reglas de descuento de club.

Sem. 9-12

Correcciones aplicadas. Nueva ronda: **38** discrepancias en **380.000** transacciones (**0,01%**). Nivel aceptado por riesgo.

Semana 13

Se invierte el sistema fuente de verdad: Drools sirve el precio; COBOL corre en sombra como validación inversa por 4 semanas.

Semana 17

COBOL se retira. Cero incidents post-cutover en precios facturados.



Cuándo NO aplicar

- ❌ Operaciones con efectos secundarios no idempotentes (enviar un email, cargar una tarjeta): el shadow run dispara el efecto dos veces.
- ❌ El sistema nuevo tiene un costo de ejecución tan alto que duplicar las operaciones es inviable presupuestalmente.
- ❌ No existe infraestructura de observabilidad para capturar y comparar resultados en tiempo real.
- ❌ Las diferencias entre los sistemas son esperadas y aceptadas por diseño (no hay 'verdad' que validar).



Criterios para aplicarlo

- ✅ El cálculo o resultado debe ser demostrable y equivalente entre los dos sistemas.
- ✅ Las operaciones son de lectura o tienen efectos idempotentes.
- ✅ Existe presupuesto para ejecutar ambos sistemas simultáneamente durante el período de validación.
- ✅ Hay un proceso formal para analizar las discrepancias encontradas por el comparador.
- ✅ El dominio es crítico (precios, inventario, scoring financiero): el costo del error en producción es alto.





Cuadro comparativo y criterios de selección

¿Cuándo elegir cada patrón?



Criterio	 Strangler Fig	 Feature Toggles	 Branch by Abstraction	 Parallel Run
 Granularidad	Funcionalidad / ruta HTTP	Feature / comportamiento	Componente / dependencia	Cálculo / dominio completo
 Dónde vive la lógica	En el proxy / router	En el sistema de toggles	En la capa de abstracción	En el comparador
 Tiempo de migración	6 – 24 meses	Días – semanas por toggle	2 – 6 meses por componente	4 – 16 semanas
 Riesgo de rollback	Bajo (re-enrutar)	Muy bajo (apagar toggle)	Medio (reactivar impl. vieja)	Muy bajo (legacy sigue activo)
 Requiere observabilidad	Media	Alta (A/B, métricas)	Baja – media	Muy alta (comparador)
 Costo de coexistencia	Alto (infraestructura doble)	Bajo	Medio	Alto (ejecución doble)
 Operaciones con side effects	Soportado	Soportado	Soportado	Problemático — requiere guards
 Caso retail típico	Migrar módulo e-commerce legacy a microservicios	Lanzar nueva UX de checkout por segmento	Reemplazar motor de cálculo interno	Validar nuevo motor de precios / scoring





Árbol de decisión: ¿qué patrón usar?



Los patrones se combinan



Strangler fig | **Feature Toggles**: migrar rutas Y controlar canary releases.



Branch by Abstraction + **Parallel Run**: nuevo componente validado con shadow traffic.



Feature Toggles + **Parallel Run**: activar el shadow run solo para el segmento de prueba.



10 Software Design Principles



Great software isn't just about making things work,
it's about creating systems that are **maintainable, scalable, and resilient**.
These fundamental design principles guide developers toward writing better code.

01



KISS (Keep It Simple, Stupid)

- ▶ The most elegant solutions are often the simplest.
- ▶ Avoid unnecessary complexity, keep code clear and concise, and focus on essential features. Remember that code is read far more often than it's written.

02



DRY (Don't Repeat Yourself)

- ▶ Every piece of knowledge in a system should have a single, unambiguous representation.

03



YAGNI (You Ain't Gonna Need It)

- ▶ Resist implementing features "just in case."
- ▶ Build what's needed today.

04



SOLID Principles

the backbone of object-oriented design:

- ▶ **Single Responsibility** – Classes should do one thing well
- ▶ **Open/Closed** – Open for extension, closed for modification
- ▶ **Liskov Substitution** – Subtypes must be substitutable for their base types
- ▶ **Interface Segregation** – Many specific interfaces beat one general interface
- ▶ **Dependency Inversion** – Depend on abstractions, not concrete implementations

05



Principle of Least Astonishment

- ▶ Software should behave as users expect.
- ▶ Consistency in terminology, conventions, and error messages creates intuitive experiences.

06



Principle of Modularity

- ▶ Well-defined, independent modules make systems easier to understand, maintain, and test.

07



Principle of Abstraction

- ▶ Hide implementation details to reduce cognitive load.
- ▶ Users of your code shouldn't need to know how it works internally, just how to use it.

08



Principle of Encapsulation

- ▶ Protect the internal state of objects from external manipulation.
- ▶ This creates more robust systems by preventing unexpected side effects.

09



Principle of Least Knowledge (Law of Demeter)

- ▶ Components should have limited knowledge of other components.
- ▶ This "need-to-know basis" approach creates more modular, flexible systems.

10



Low Coupling & High Cohesion

- ▶ Minimize dependencies between modules while ensuring each module has a clear, unified purpose.
- ▶ This balance makes systems more maintainable and adaptable.



Grandes arquitectos de software no solo construyen sistemas:
multiplican valor, crean impacto y dejan legado.



Documentación y Deuda Técnica

Documentar hoy, menos incertidumbre mañana

Una buena documentación transmite conocimiento, reduce la incertidumbre y facilita decisiones informadas.



¿Qué es?

La documentación es el conjunto de artefactos que registran el **qué, por qué y cómo** de un sistema o producto.

En el contexto del proyecto, una buena documentación ayuda a mantener la **coherencia con las decisiones de arquitectura**, los supuestos y los entregables.



Buenas prácticas

- 1 **Usa plantillas comunes.**
Aseguran consistencia y calidad.
- 2 **Mantén la documentación actualizada.**
Evoluciona junto con el sistema.
- 3 **Sé claro y conciso.**
Enfócate en lo esencial.
- 4 **Vincula la documentación al código.**
(README, diagramas, ADR, etc.).
- 5 **Hazla accesible.**
En un repositorio central para todo el equipo.



¿Por qué importa?



Acelera el onboarding y la transferencia de conocimiento.



Reduce errores y retrabajo.



Facilita el mantenimiento y la evolución.



Permite justificar decisiones y gestionar la deuda técnica.



Hace visible la arquitectura, los supuestos y los trade-offs.



Señales de alerta



Documentación desactualizada o inexistente.



Dificultad para entender cómo funciona la solución.



Conocimiento concentrado en pocas personas.



Inconsistencias entre lo documentado y lo implementado.



Falta de evidencia de pruebas, despliegue u observabilidad.

Tipos de documentación y su relación con deuda técnica

Tipo de documentación	Valor / Beneficio	Riesgo si falta (deuda técnica)
 Documentación de arquitectura (vistas, decisiones, supuestos)	Alinea al equipo, facilita decisiones y evolución.	Soluciones inconsistentes, dificultad para escalar, pérdida de contexto.
 Documentación de código (README, comentarios, guías)	Facilita el entendimiento y mantenimiento.	Curva de aprendizaje alta, mayor tiempo de análisis y errores.
 Documentación operativa (despliegue, configuración, runbooks)	Operación estable y recuperación rápida.	Incidentes más largos, mayor impacto en el negocio.
 Documentación de pruebas y observabilidad	Evidencia calidad y comportamiento real.	Falta de confianza, dificultad para validar cambios y detectar fallos.
 Registro de decisiones (ADR - Architecture Decision Records)	Explicita alternativas, trade-offs y justificación.	Se repiten discusiones, se toman decisiones sin contexto, aumenta la deuda técnica.



Mensaje clave

Documentar bien es diseñar para el futuro.

Menos suposiciones, más claridad. Menos riesgo, más valor.



Usa ADR

Registra decisiones, su contexto, alternativas, trade-offs y consecuencias. Te ayudará a gestionar la deuda técnica.



Trabajo en equipo

La documentación es una responsabilidad compartida y parte de la calidad del proyecto.



Deuda Técnica

Atajos de hoy, costos de mañana



¿Qué es?

La deuda técnica es el costo implícito que asumimos hoy al tomar atajos o decisiones que optimizan el corto plazo, pero que generan mayor esfuerzo, riesgo y complejidad en el futuro.

"Como la deuda financiera, permite avanzar hoy, pero acumula intereses si no se gestiona a tiempo."

Ward Cunningham (1992)

¿Por qué aparece?

Algunas causas comunes:



Presión de tiempo
"Funciona, lo vemos después".



Falta de conocimiento
o experiencia.



Requerimientos cambiantes.



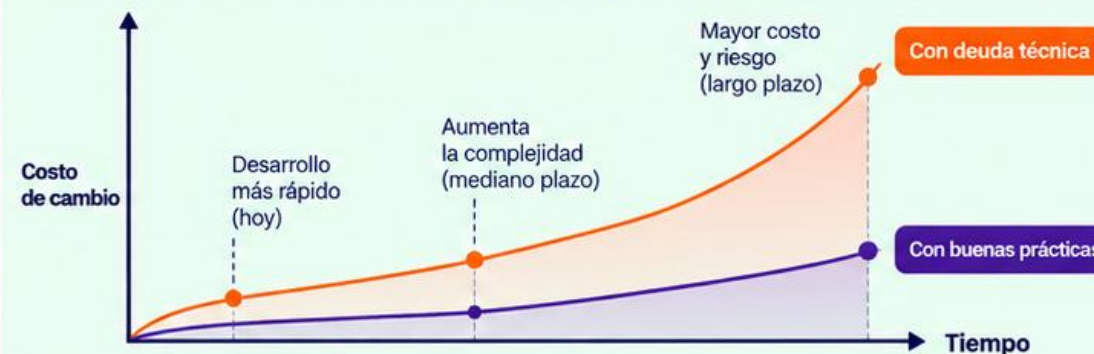
Falta de estandarización
y buenas prácticas.



Compromisos de negocio
y entrega rápida.

Impacto en el tiempo

Pequeñas decisiones hoy pueden generar grandes costos mañana.



Consecuencias

Si no se gestiona, la deuda técnica:



Aumenta los
costos de desarrollo
y mantenimiento.



Retrasa la entrega
de nuevas
funcionalidades.



Incrementa el riesgo
de fallos en
producción.



Reduce la calidad
del software y la
satisfacción del usuario.



Dificulta la
escalabilidad y
la evolución del sistema.

¿Cómo gestionarla?

La clave está en ser conscientes y tener una estrategia:



Identificar
y priorizar.



Medir su
impacto.



Refactorizar
de forma
planificada.



Aplicar buenas
prácticas
(arquitectura,
SOLID, revisiones).



Incluirla en la
toma de decisiones
del proyecto.



Mensaje clave

La deuda técnica no es mala, pero ignorarla sí.

Gestionarla bien hoy, es entregar valor sostenible mañana.

"El software también tiene intereses.

La diferencia está en si decides pagarlos o dejarlos crecer."

Deuda Técnica

Decisiones de hoy que tienen un costo en el futuro



Ward Cunningham (1992): la deuda técnica es el costo implícito del retrabajo futuro causado por elegir una solución fácil ahora en lugar de una mejor que tomaría más tiempo. Como la deuda financiera, acumula intereses: cuanto más se pospone el pago, mayor el costo total.



Es normal incurrir en deuda técnica. Lo importante es ser conscientes, medirla, gestionarla y pagarla de forma oportuna.

Tipos de deuda técnica

No toda la deuda técnica es mala. Lo importante es reconocerla y gestionarla.

Deliberada Prudente



Decisión consciente de tomar un atajo conocido para cumplir un objetivo de corto plazo (ej. validar una hipótesis o llegar a un deadline).



Es válida si se documenta, tiene un plan de pago y se monitorea.

Deliberada Imprudente



Se toma el atajo sabiendo el costo, pero sin un plan de pago. "No tenemos tiempo para diseñarlo bien". La más dañina a largo plazo.



Genera riesgos y costos significativos si no se aborda.

Inadvertida Prudente



Se descubre retrospectivamente: "Ahora entendemos mejor el problema". Surge de la evolución del conocimiento del equipo y del contexto.



Se puede gestionar con aprendizaje y refactorización planificada.

Inadvertida Imprudente



El equipo no sabe que está generando deuda. Falta de conocimiento sobre patrones, principios SOLID, arquitectura limpia, o las implicaciones de la decisión.



Se previene con formación, revisiones de arquitectura y buenas prácticas.

Impacto de la deuda técnica

Si no se gestiona, la deuda técnica:



Aumenta los costos de desarrollo y mantenimiento.



Retrasa la entrega de nuevas funcionalidades.



Incrementa el riesgo de fallos en producción.



Reduce la calidad del software y la satisfacción del usuario.



Dificulta la escalabilidad y la evolución del sistema.



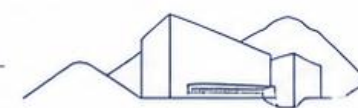
Mensaje clave

La deuda técnica es parte del desarrollo de software. Lo importante es tomar **decisiones informadas**, **ser coherentes** con la arquitectura, **documentar** los supuestos y **establecer un plan** para gestionarla.

DECISIONES HOY

MENOS DEUDA

MÁS VALOR



Acoplamiento y cohesión

Conceptos clave para diseñar sistemas de calidad



Acoplamiento: la interdependencia entre módulos de un sistema. Un **alto acoplamiento** significa que los módulos están muy interconectados y dependen unos de otros para funcionar. Un **bajo acoplamiento** significa que los módulos son independientes y no dependen mucho unos de otros.



Cohesión: la cohesión se refiere a la fuerza de la relación entre los elementos de un módulo. Un módulo con **alta cohesión** tiene elementos que están estrechamente relacionados y trabajan juntos para lograr un objetivo común. Un módulo con **baja cohesión** tiene elementos que están débilmente relacionados y no trabajan juntos de manera efectiva.



Relación entre acoplamiento y cohesión:

En general, un **bajo acoplamiento** es deseable porque significa que los módulos son independientes y fáciles de modificar y mantener. Una **alta cohesión** también es deseable porque significa que los módulos son fáciles de entender y mantener.





02 · ACOPLAMIENTO Y COHESIÓN

Profundidad técnica

ACOPLAMIENTO — Cuánto sabe un módulo del interior de otro

Contenido (máximo)

Módulo A modifica directamente las variables internas de B. La arquitectura más frágil posible.

Control

A le pasa a B un flag que controla su flujo interno (p.ej. un boolean 'isFast').

Externo / de datos comunes

A y B comparten una estructura global (base de datos, variable global, archivo config compartido).

De sello (stamp)

A pasa a B una estructura compleja aunque B solo necesite un campo.

De datos (ideal)

A y B se comunican solo a través de parámetros simples y bien definidos. Máximo desacoplamiento.

COHESIÓN — Cuán relacionadas están las responsabilidades dentro de un módulo

Funcional (ideal)

Todo en el módulo contribuye a una única tarea bien definida. `PaymentProcessor` solo procesa pagos.

Secuencial

La salida de una función es la entrada de la siguiente. Aceptable pero frágil si el orden cambia.

Comunicacional

Las funciones operan sobre los mismos datos pero hacen cosas distintas. Frontera borrosa.

Lógica

El módulo agrupa funciones similares controladas por un flag (`processAnything(type)`). Anti-patrón.

Coincidental (mínima)

Las funciones están juntas por conveniencia histórica, sin relación real. `Utils.java` con 3.000 líneas.

Ley de Constantine: un buen diseño maximiza la cohesión interna y minimiza el acoplamiento externo. Son fuerzas opuestas que deben gestionarse simultáneamente.



Ventanas



Strangler



Datos Primero



Parallel Run

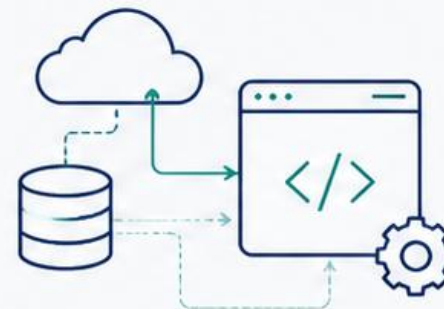


ACL

Arquitectura • Modernización Evolutiva

Migración de Sistemas

Ventanas de migración • Strangler Fig • Datos Primero
Parallel Run • Anti-Corruption Layer



Strangler Fig



Datos Primero



Parallel Run



Anti-Corruption Layer



Casos reales:

banca, retail, salud,
manufactura y gobierno



Cada patrón incluye:

cuándo aplicarlo • cómo implementarlo
ejemplos • anti-patrones





01 · Ventanas de migración

— cuándo y cuánto cambiar

Framework



Una ventana de migración es el periodo óptimo para ejecutar un cambio sin maximizar el riesgo operacional. Elegir mal la ventana es tan costoso como elegir mal el patrón: el mejor Strangler Fig lanzado en el pico de temporada crea más deuda que la que elimina.



Eje TEMPORAL

¿Cuándo en el año / sprint?



Temporada baja

Retail: ene-feb post-navidad. Banca: fuera de cierres de mes. Manufactura: mantenimiento programado de planta.



Freeze regulatorio

Evitar 2 semanas antes/después de auditorías INVIMA, DIAN, SFC. Los cambios en esas ventanas anulan la evidencia de control.



Después del release anterior

Esperar 2 sprints de estabilización post-deploy antes de iniciar migración. Los bugs latentes del release anterior se solapan con los de la migración.



Eje DE ALCANCE

¿Cuánto cambiar de una vez?



Regla del 10%

No migrar más del 10% de la base de código en un solo sprint de migración. Más del 10% hace inmanejable el rollback y el análisis de causa raíz.



Slice vertical

Migrar una funcionalidad completa end-to-end (UI→API→DB) en lugar de una capa horizontal. Los slices verticales son demostrables al negocio.



Dominio cerrado

El primer módulo a migrar debe tener el menor número de dependencias entrantes (Ca más bajo). Minimiza el blast radius si algo falla.



Eje DE RIESGO

¿Cuánto riesgo puedo absorber?



RTO/RPO como límite

Si el sistema tiene RTO < 4h, la migración necesita un rollback automático preconfigurado. Sin él, la ventana no existe: siempre es el momento equivocado.



Capacidad de reversión

Un cambio es migracionalmente seguro solo si puede revertirse en menos tiempo del que tomó desplegarse. Sin reversibilidad, no hay ventana.



Cobertura de pruebas como prerequisite

Coverage < 40% en el módulo a migrar = no hay ventana. Cada punto de coverage por debajo del 70% agrega 1 semana de riesgo efectivo.



Regla práctica: una ventana de migración es válida cuando los tres ejes son favorables simultáneamente. Si uno falla, posponer — el costo de esperar siempre es menor al costo de un rollback de emergencia en producción.



El sistema nuevo envuelve progresivamente al legacy ruta a ruta. Un proxy enruta tráfico: el legacy recibe lo que no ha sido migrado; el nuevo sistema lo que sí. **La coexistencia es la norma**, no la excepción.

Arquitectura de ejecución



Secuencia de implementación

- 1** **Instalar el proxy sin tocar el legacy:**
AWS API Gateway, Kong, NGINX o un BFF. Configurar que el 100% del tráfico siga al legacy. Validar en producción que el proxy no introduce latencia > 20ms.
- 2** **Seleccionar el primer slice (mínimo acoplamiento):**
Elegir la ruta con Ca=0 (nadie más llama a ella) y Ce bajo. Suele ser una ruta de lectura de catálogo o de reporting, no una transaccional.
- 3** **Canary release controlado:**
5% → 20% → 50% → 100% del tráfico al nuevo servicio en esa ruta. Ventana entre cada incremento: 48–72h. Monitorear error rate, p99 latencia y consistencia de datos.
- 4** **Retirar la ruta del legacy:**
Una vez 100% migrada y estable por 2 semanas, eliminar la ruta del legacy. No "comentar código": borrar. El legacy debe encogerse activamente.



No aplicar cuando: el sistema no tiene interfaces HTTP definidas · plazo < 3 meses · equipo sin experiencia en API Gateways



Aplicar cuando: sistema grande con múltiples rutas · equipo puede mantener el legacy mientras migra · +6 meses disponibles



Caso Real — Banco Itaú Brasil: migración del Core Bancario



Contexto: Monolito COBOL de 40 años procesando 12M transacciones/día. Sin downtime posible.



Proxy AWS API Gateway + Lambda authorizer instalado en 3 semanas sin tocar el COBOL.



Primer slice: endpoint de consulta de saldo (solo lectura, Ca=0). Migrado en 6 semanas. Canary 5%→100%.



36 meses después: 78% del tráfico en microservicios Java. COBOL solo atiende módulo de liquidación nocturna.



Resultado: 0 downtime registrado · latencia p99 mejoró 40% · equipo COBOL de 45 personas reducido a 6.



Anti-patrón evitado: NO reescribir la lógica de negocio del COBOL. Solo mover, no reinterpretar.



Migrar primero el modelo de datos hacia su forma objetivo, desacoplando la lógica de la persistencia. Las aplicaciones se adaptan al nuevo esquema de forma gradual mediante vistas de compatibilidad y APIs de traducción.

¿Por qué migrar los datos primero?



Los datos son el activo con mayor costo de migración y mayor riesgo de pérdida. Si los datos se migran al final, el sistema nuevo corre con datos del sistema viejo hasta el último momento — el período de máxima fragilidad.



La lógica de negocio es más fácil de migrar cuando el modelo de datos ya está estable. Al revés (lógica primero, datos después), la lógica se escribe dos veces: una para el modelo viejo y otra para el nuevo.

Fases de implementación

F1



Duplicar el esquema

Crear el nuevo esquema en paralelo. Las tablas antiguas siguen activas. No eliminar ningún campo del esquema viejo todavía.

F2



Sincronización dual

Event sourcing o triggers escriben en ambos esquemas simultáneamente. Validar consistencia en tiempo real con el comparador.

F3



Migración de datos históricos

ETL incremental en lotes nocturnos. Nunca migrar en batch único: dividir en chunks de 10K-100K registros con puntos de control.

F4



Cutover de aplicaciones

Redirigir las aplicaciones al nuevo esquema. Las vistas de compatibilidad mantienen el contrato con el esquema viejo. Eliminar después.



Caso Real — Amazon: migración de Oracle a DynamoDB + Aurora



Contexto: Amazon.com tenía su catálogo de productos en Oracle (2.5 TB, 75M SKUs). La capa de negocio (Java/Perl) asumía el esquema relacional en 120 stored procedures.



Decisión Datos Primero: Crear el nuevo esquema en DynamoDB (NoSQL) mientras Oracle seguía activo. Un servicio de sincronización dual-write mantenía ambos en sync.



Vista de compatibilidad: ProductCompatibilityView exponía una API REST con la misma semántica del esquema Oracle. Los 120 stored procedures se reemplazaron gradualmente usando esta API.



Resultado (18 meses): Oracle completamente retirado. Query latency: 300ms → 4ms. Costo infra: -70%. La lógica de negocio se migró sin una sola reescritura 'big bang'.



Lección clave: Migrar primero los datos de producto de baja volatilidad (sin transacciones pendientes) antes de datos de órdenes y pagos. La secuencia de los datos dentro del patrón importa tanto como el patrón mismo.



Anti-patrón crítico: migrar datos y lógica simultáneamente. Cuando algo falla (y fallará), es imposible determinar si el problema es del esquema nuevo, de la lógica o de la sincronización. Siempre separar las fases.





04 · Parallel Run

— Ejecución en Sombra

Patrón 3



El sistema nuevo ejecuta en **modo sombra (shadow)**: recibe las mismas peticiones que el legacy, procesa en paralelo, pero sus resultados **NO se sirven al cliente**. El comparador automático analiza discrepancias y las clasifica por severidad.

Arquitectura del Parallel Run



Clasificación de discrepancias

✓ Nivel 0 - Equivalente	Resultados idénticos. La migración avanza.	→ Continuar canary
i Nivel 1 - Divergencia aceptada	Diferencia en formato o precisión pero semánticamente equivalente. Documentar.	→ Documentar, no bloquear
! Nivel 2 - Divergencia funcional	El nuevo sistema produce resultados diferentes pero plausibles. Investigar.	→ Pausar canary · Analizar
✗ Nivel 3 - Error crítico	El nuevo sistema produce excepciones, timeouts o resultados incorrectos.	→ Detener · Rollback · Fix



Caso Real — GitHub: migración de MySQL a MySQL+Vitess (GitHub Scientist)



Contexto: GitHub tenía un monolito Ruby on Rails con MySQL. Al llegar a 50M repos necesitaban sharding horizontal.



Herramienta: GitHub Scientist (librería open source) implementa Parallel Run nativo: candidate branch corre en sombra y el control branch sirve la respuesta.



Proceso: Cada endpoint crítico (pull requests, issues, commits) corrió en shadow mode 4-8 semanas antes del cutover. Comparador analizada mismatches en tiempo real.



Métrica de éxito: Discrepancias Nivel 1 < 0.01% durante 2 semanas consecutivas → go para cutover. 3 endpoints requirieron hotfixes antes de alcanzar el umbral.



Resultado: Cero incidentes de datos en producción post-migración. El shadow mode detectó 14 edge cases que los tests unitarios no habían capturado.



Restricción crítica: operaciones con efectos secundarios (emails, pagos, notificaciones) deben bloquearse en el shadow path o el Parallel Run dispara el efecto dos veces.





05 · Anti-Corruption Layer

— Aislamiento del Legacy

Patrón 4



Capa de traducción bidireccional que impide que el modelo de dominio del sistema nuevo se 'corrompa' adoptando los conceptos y estructuras del sistema legado.

El ACL traduce el lenguaje del legacy al lenguaje del sistema nuevo — **nunca al revés.**

¿Cuándo es necesario el ACL?



El sistema legado usa un **modelo de datos profundamente diferente** al del sistema nuevo (ej.: SAP IDOC vs REST JSON).



El equipo **no puede modificar** el sistema legado (sistema de un tercero, black box, contrato cerrado).



El lenguaje ubiquitous del dominio del nuevo sistema **no puede 'traducirse'** limpiamente a los conceptos del legacy.

Estructura interna del ACL



Caso Real — Falabella Colombia: migración del ERP Oracle ATG



Contexto: Oracle ATG como ERP de e-commerce con modelo propio de 'Cart', 'Linnettem', 'Promotion'. El equipo nuevo diseñó un sistema basado en DDD con 'Carrito', 'ItemPedido', 'Descuento'.



Sin ACL (lo que NO hicieron): El modelo nuevo habría importado los conceptos de ATG: Linnettem con 23 campos heredados irrelevantes para el nuevo dominio.



ACL implementado: CartTranslationService: convierte un ATG Linnettem a un ItemPedido del nuevo modelo, eliminando 19 campos irrelevantes y renombrando 4 entidades clave.



Facade de legacy: ATGCartFacade expone solo 8 de los 47 endpoints del ATG que el nuevo sistema necesita. Los otros 39 quedan ocultos.



Resultado: El nuevo sistema de carrito se desarrolló en 6 meses con un modelo de dominio limpio. Ningún desarrollador del equipo nuevo tuvo que leer documentación de ATG.



Métricas: Líneas de código de traducción: 4.200. Bugs de integración en producción post-go-live: 2 (ambos en el Translator, no en la lógica de negocio).



Anti-patrón del ACL: que el ACL sea tan complejo que se convierta en el nuevo sistema legado.

Regla: si el ACL supera el 15% del código del sistema nuevo, la arquitectura de dominio del sistema nuevo está mal diseñada.





06 · Cuadro comparativo y criterios de selección

Patrón 4

Criterio	Strangler Fig	Datos Primero	Parallel Run	Anti-Corruption Layer
 Riesgo operacional	Bajo (rollback = re-enrutar)	Medio (sync dual-write)	Muy bajo (legacy sigue activo)	Bajo (legacy no se modifica)
 Duración típica	6 – 24 meses	3 – 12 meses	4 – 16 semanas por módulo	Variable (vive durante toda la migración)
 Prerequisito técnico	API Gateway + observabilidad	Dual-write + ETL + vistas compat.	Shadow router + comparador auto.	Definición del modelo de dominio nuevo
 Side effects de negocio	Soportado con cuidado	Soportado	Problemático — requiere guardrails	No aplica (solo traducción)
 Costo de coexistencia	Alto (infra doble)	Medio (BD dual)	Medio (CPU doble)	Bajo (capa de traducción)
 Señal de uso	Sistema grande con rutas HTTP	Modelo de datos a transformar	Lógica de cálculo crítica a validar	Integración con sistema de tercero / black box
 Caso real citado	Itaú Brasil — COBOL	Amazon — Oracle → DynamoDB	GitHub — MySQL → Vitess	Falabella — Oracle ATG



Los patrones se combinan: Strangler Fig (enruta) + Datos Primero (migra BD) + ACL (aísla el modelo) es la **triada más usada** en migraciones enterprise de más de 12 meses. El Parallel Run se añade en los módulos de mayor riesgo calculatorio.

Strangler Fig
(enruta)

+

Datos Primero
(migra BD)

+

ACL
(aísla el modelo)

+

Parallel Run
(valida)



Principios universales de migración



La migración exitosa no es un evento: es un proceso continuo con puntos de control definidos y criterios de avance explícitos.



Nunca migrar lógica y datos al mismo tiempo en el mismo sprint. Siempre secuenciar: datos primero, lógica después — o lógica con ACL, datos con su propio plan.



El rollback es una feature, no un plan de contingencia. Si no está diseñado y probado antes de iniciar la migración, la migración no puede comenzar.



Medir antes de migrar: I, A, CBO, cobertura de tests. Un componente con cobertura < 40% no tiene ventana de migración hasta que se instrumenta.



La migración termina cuando el sistema legacy es retirado, no cuando el sistema nuevo está en producción. La coexistencia tiene un costo que escala con el tiempo.



Anti-patrones que matan migraciones



Big Bang oculto

Llamar 'incremental' a una migración que tiene un cutover único de todo el sistema.



Proxy que transforma

Un API Gateway que implementa lógica de negocio. Se convierte en el nuevo legado en 6 meses.



ACL permanente

La capa de anti-corrupción que nunca se retira porque nadie rediseña el modelo de dominio nuevo.



Datos sin linaje

Migrar datos sin trazabilidad de origen. Cuando hay una discrepancia, es imposible determinar en qué momento ocurrió.



Migración sin negocio

El negocio no conoce el plan de migración. Cada cambio visible es una sorpresa. El primer incidente cancela el proyecto.



Referencias: M. Fowler — *StranglerFigApplication* • E. Evans — *DDD (Anti-Corruption Layer)* • GitHub Scientist (github.com/github/scientist)
AWS — *Database Migration Patterns* • O'Reilly — *Software Architecture Patterns*



El Reto de la Gestión Tradicional



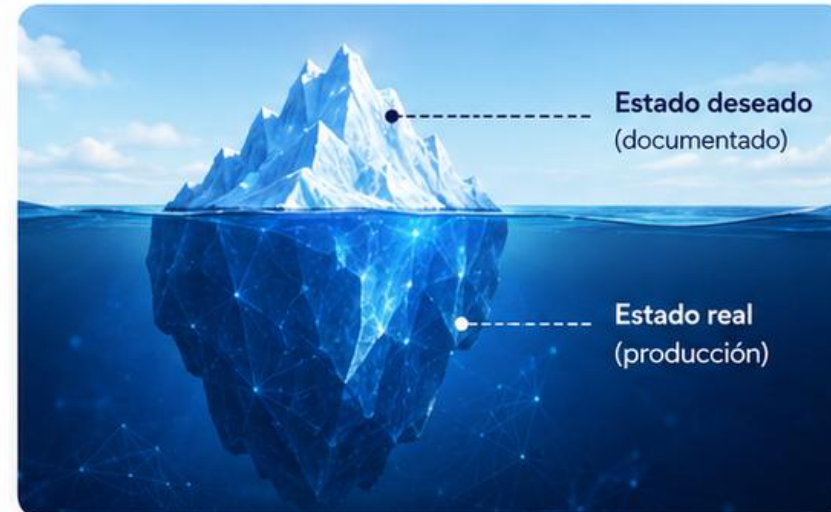
Infraestructura "Snowflake"

Configuraciones manuales únicas e irreproducibles. La falta de documentación genera silos de conocimiento y miedo al cambio.



Configuration Drift

La divergencia entre el estado deseado y el real en producción. Sin auditoría clara, los incidentes son difíciles de diagnosticar.



Infraestructura como Código (IaC)

IaC es el proceso de gestionar y aprovisionar infraestructura mediante archivos de definición legibles por máquina.

- ✓ **Declarativo:** Defines el "qué", no el "cómo".
- ✓ **Idempotencia:** Ejecuciones repetidas dan el mismo resultado.
- ✓ **Inmutabilidad:** Reemplazar en lugar de modificar.



La transición de configuraciones manuales a infraestructura como código transforma la gestión operativa en un proceso controlado, repetible y seguro.



Herramientas: El Stack Tecnológico



El uso de Infraestructura como Código (IaC) permite gestionar la infraestructura de forma declarativa, versionada y automatizada.

Terraform
Estándar industrial para el aprovisionamiento multi-cloud declarativo.

Pulumi
Infraestructura usando lenguajes reales (TS, Python, Go).

Ansible
Gestión de configuración y automatización de tareas en SO.

Crossplane
Control plane nativo de Kubernetes para recursos cloud.

Comparativo rápido

Criterio	Terraform	Pulumi	Ansible	Crossplane
Enfoque	Aprovisionamiento multi-cloud	Aprovisionamiento con lenguajes	Configuración y automatización	Control plane para plataformas
Lenguaje	HCL (declarativo)	TS, Python, Go, .NET, Java (imperativo)	YAML (declarativo)	YAML (declarativo - Kubernetes)
Cobertura Cloud	Multi-cloud (AWS, Azure, GCP, etc.)	Multi-cloud (AWS, Azure, GCP, etc.)	On-prem, hybrid, multi-cloud	Cualquier proveedor (vía CRDs)
Caso de uso ideal	Plataformas, landing zones, redes, servicios cloud	Equipos de desarrollo, lógica compleja, integraciones	Configuración de servidores, aplicaciones, orquestación	Plataformas internas, self-service en Kubernetes
Curva de aprendizaje	Media	Media (menor si ya conoces el lenguaje)	Baja	Alta (conceptos de Kubernetes y CRDs)
Fortalezas clave	Madurez, ecosistema, modules, state management	Reutilización de código, testing, abstracciones poderosas	Simplicidad, agente-less, excelente para ops	Extensibilidad, control unificado, nativo de Kubernetes

Beneficios de IaC
para la organización

Consistencia y repetibilidad

Visibilidad y auditoría

Entrega más rápida y segura

Colaboración entre equipos

Reducción de costos y riesgos

GitOps: Operación vía Git

GitOps extiende IaC utilizando **Git como la Fuente Única de Verdad** para todo el sistema.

Los cambios no se 'empujan' manualmente; un agente interno reconcilia el estado del clúster con lo declarado en el repositorio mediante un **loop infinito**.



Declarativo y consistente

El estado deseado se define en archivos versionados. Trazabilidad completa.



Automatización continua

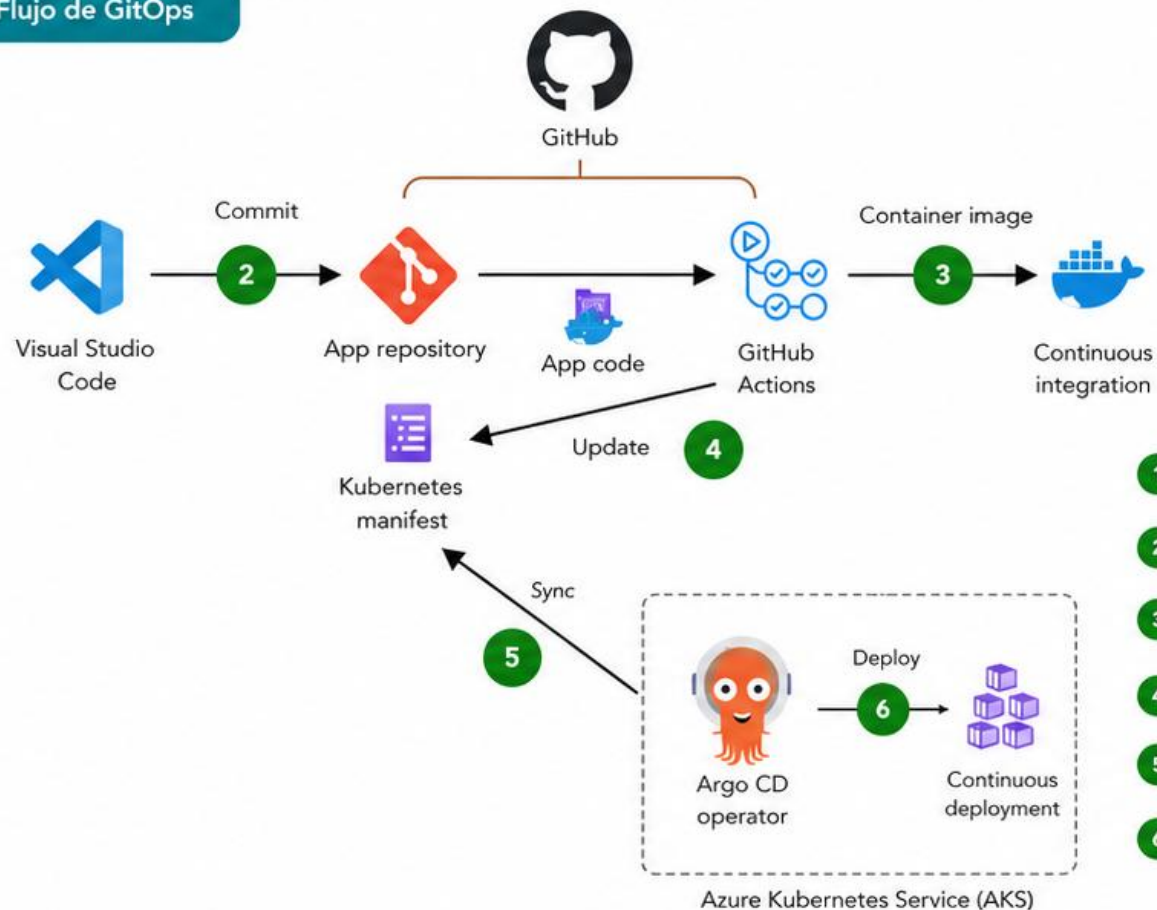
Sin intervención manual. Reconciliación automática ante cualquier drift.



Auditable y seguro

Historial de cambios, revisiones y aprobaciones mediante Pull Requests.

Flujo de GitOps



- 1 Desarrollador realiza cambios en el código
- 2 Commit y push al repositorio
- 3 CI construye la imagen de contenedor
- 4 Actualiza los manifiestos de Kubernetes
- 5 Argo CD sincroniza el estado deseado
- 6 Despliegue continuo en el clúster



¿Por qué
GitOps?



Todo es código
Versionado y
reproducible



Fuente única
de verdad
GitHub como
origen confiable



Reconciliación
automática
Reduce drift y
errores humanos



Observabilidad
completa
Mejor visibilidad
del estado



Seguridad
integrada
Auditoría, permisos
y aprobaciones

Rúbrica Entregable 3

De la arquitectura al valor real: construimos, medimos, probamos y aprendemos



El Entregable 3 evalúa una solución que **funciona, se observa, resiste fallos, aplica patrones** y es **coherente** con todo lo definido.



40%

1. Aplicación funcionando

La aplicación se ejecuta de forma correcta y cumple con los escenarios definidos.

Se valora:

- Funcionalidades principales operativas.
- Despliegue en el entorno definido.
- Experiencia de usuario estable.
- Evidencias de ejecución (video/demo, accesos, ejemplos de uso).



20%

2. Observabilidad

3 métricas de negocio y 3 métricas técnicas funcionando de forma real.

Se valora:

- Definición y justificación de las métricas.
- Recolección y visualización en tiempo real.
- Uso de herramientas de observabilidad.
- Evidencias (dashboards, logs, trazas, alertas).



30%

3. Simulación y análisis de fallos

Se simulan 4 fallos y se analiza el comportamiento de la aplicación.

Se valora:

- Ejecución de 4 escenarios de fallo (tipos diferentes).
- Análisis del comportamiento del sistema (métricas, recuperación, degradación, etc.).
- Conclusiones y aprendizajes.
- Evidencias (scripts, registros, capturas, dashboards).



10%

4. Patrones utilizados

Explicación de los patrones de diseño/arquitectura utilizados en la solución.

Se valora:

- Identificación de patrones.
- Justificación de su uso.
- Relación con atributos de calidad (escalabilidad, resiliencia, mantenibilidad, etc.).
- Evidencias en la implementación y en la documentación.



+10%

5. Autoevaluación (valor adicional)

Reflexión crítica del equipo sobre el resultado, aprendizajes y oportunidades de mejora.

Se valora:

- Análisis honesto y fundamentado.
- Identificación de logros, dificultades y mejoras.
- Propuestas concretas de evolución.
- Coherencia con la evidencia del proyecto.



Hasta -10%

Coherencia (resta puntos)

El resultado debe ser coherente con las decisiones de arquitectura, los supuestos, el alcance y los entregables anteriores.

Se descuentan puntos si:

- La solución implementada no es coherente con las decisiones de arquitectura y no se han actualizado los supuestos/decisiones (ADR).
- Faltan entregables del Entregable 2 (p. ej. ejecución de pruebas unitarias, información de observabilidad, modelo de dominio, diagramas) o no son consistentes con la solución implementada.
- La documentación no refleja el estado real del proyecto.



Evidencias de entrega

Código, despliegue, dashboards, registros, documentación, video demo, entre otros.



Evaluación

Cada criterio se califica según los niveles de desempeño definidos en la rúbrica detallada.



Resultado del entregable

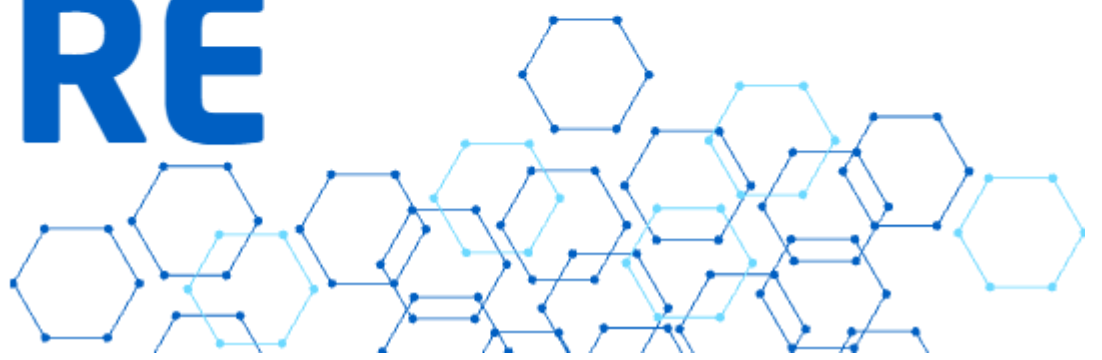
La calificación se calcula sobre 100 puntos, se aplica el descuento por coherencia (si aplica) y luego se pondera al 30% del proyecto.

Fórmula de calificación

$$\text{Nota Entregable 3} = \left(\frac{\text{Puntos base}}{(0 - 100)} + \frac{\text{Puntos adicionales}}{(0 - 10)} - \frac{\text{Puntos de descuento}}{(0 - 10)} \right) \times 30\%$$



ALGUNOS PATRONES DE DESARROLLO DE SOFTWARE



Patrones de Diseño

Soluciones probadas para problemas recurrentes

“Un patrón de diseño captura la experiencia de expertos y la hace reutilizable.”
Erich Gamma (GoF)

 ¿Qué son?

Los patrones de diseño son **soluciones generales y reutilizables** a problemas comunes en el diseño de software.

No son código específico, sino plantillas conceptuales que guían cómo estructurar la solución de forma flexible, mantenible y escalable.

 ¿Por qué usarlos?

- ✓ Reutilizan conocimiento probado.
- ✓ Mejoran la mantenibilidad y escalabilidad.
- ✓ Reducen la complejidad.
- ✓ Facilitan la comunicación en el equipo.
- ✓ Promueven buenas prácticas de diseño.
- ✓ Ayudan a resolver problemas recurrentes más rápido y con menor riesgo.

 **Categorías principales**
Los patrones de diseño se agrupan según el propósito que cumplen:

 Creacionales Controlan la creación de objetos.	Ejemplos: Singleton, Factory Method, Builder.
 Estructurales Componen clases y objetos.	Ejemplos: Adapter, Decorator, Facade.
 De comportamiento Definen la comunicación entre objetos.	Ejemplos: Observer, Strategy, Command.

 **Buenas prácticas**

- ✓ Usa patrones cuando el problema sea recurrente.
- ✓ Selecciona el patrón según el contexto y los requisitos.
- ✓ Evita el sobre-diseño.
- ✓ Documenta el patrón y su justificación.
- ✓ Combínalos con principios SOLID y arquitectura limpia.

 **Ejemplos comunes**
Patrones que hacen la diferencia en la práctica:

 **Singleton**
Garantiza una única instancia de una clase.

Una sola instancia, acceso global controlado.

 **Factory Method**
Delega la creación de objetos a subclases.

Creación flexible de objetos.

 **Observer**
Define una dependencia uno a muchos.

Notifica cambios a múltiples interesados.

 **Strategy**
Permite intercambiar algoritmos en tiempo de ejecución.

Distintas estrategias, mismo contexto.

 **Adapter**
Permite que interfaces incompatibles trabajen juntas.

Conecta lo que no encaja por defecto.

 **Más que código**
Usar patrones de diseño es una forma de pensar en soluciones elegantes, reutilizables y alineadas con una arquitectura de calidad.

“Los buenos diseñadores resuelven problemas. Los grandes, los resuelven una y otra vez.”

 **Mensaje clave**
Los patrones de diseño son una inversión en la calidad del software.
Más claridad hoy, sistemas más sostenibles mañana.

 Código más limpio

 Equipos más eficientes

 Soluciones escalables

Patrones de Diseño

Soluciones probadas para problemas reales

Los patrones de diseño son **soluciones generales y reutilizables** que ayudan a resolver problemas comunes en el diseño de software, haciendo el código más flexible, mantenible y escalable.



Un buen diseño reutiliza lo probado para construir mejores soluciones.

Erich Gamma (GoF)



Patrón	Idea central	Ejemplo de la vida real	Cómo se refleja en software
 Singleton	Existe una única instancia compartida de algo.	 Control de acceso a un edificio: existe un único sistema central que mantiene el registro de quién entra y sale. Todos los puntos de acceso consultan ese mismo registro.	Una única instancia de configuración, gestor de logs o conexión compartida.
 Factory Method	Delegar la creación de un objeto según la necesidad.	 Máquina de bebidas: el usuario pide café, té o chocolate y la máquina decide qué objeto preparar. El usuario no necesita conocer cómo se construye cada bebida.	NotificationFactory puede crear Email, SMS o Push según el tipo solicitado.
 Observer	Un cambio en un objeto notifica automáticamente a varios interesados.	 Suscripción a un canal de YouTube: cuando se publica un video, todos los suscriptores reciben una notificación. El canal no necesita contactar manualmente a cada persona.	Cuando cambia un pedido, se notifican automáticamente inventario, facturación, cliente y logística.
 Strategy	Permite intercambiar diferentes algoritmos para resolver el mismo problema.	 Aplicación de navegación: para llegar al mismo destino puede elegir ruta en carro, transporte público, bicicleta o caminando. El objetivo es el mismo, pero cambia la estrategia.	Un sistema de pagos puede utilizar PagoConTarjeta, PagoConPSE, PagoConPayPal, etc.
 Adapter	Hace compatibles dos interfaces que originalmente no pueden comunicarse directamente.	 Adaptador eléctrico: un dispositivo con enchufe europeo puede conectarse a un tomacorriente colombiano mediante un adaptador. Ninguno de los dos dispositivos necesita modificarse.	Una aplicación puede usar una interfaz propia para comunicarse con una API externa cuyo formato es diferente.



Mensaje clave

Los patrones de diseño encapsulan experiencia, reducen la complejidad y facilitan la evolución del software.

Usar patrones no es complicar, es diseñar con propósito.

Código más limpio.
Soluciones más duraderas.
Un mejor futuro.



¿Qué es la idempotencia?

El mismo resultado, sin importar cuántas veces se ejecute.

1. Definición



Una operación es **idempotente** cuando, al ejecutarse una o varias veces con los mismos parámetros, produce el mismo efecto y el mismo resultado en el sistema.



“Ejecutar una vez o muchas veces, el resultado es el mismo.”

2. Ejemplos concretos



Operación idempotente
PUT /users/123

Actualiza el usuario 123 con la misma información. Si se ejecuta varias veces, el usuario queda igual.

PUT /users/123
{ "nombre": "Ana" }

PUT /users/123
{ "nombre": "Ana" }

PUT /users/123
{ "nombre": "Ana" }



Usuario 123
nombre = Ana

Resultado final:
siempre el mismo



Operación NO idempotente
POST /orders

Crea un nuevo pedido. Si se ejecuta varias veces, se crean varios pedidos diferentes.

POST /orders
{ "producto": "Libro" }

POST /orders
{ "producto": "Libro" }

POST /orders
{ "producto": "Libro" }



Resultado final:
varios pedidos

3. ¿Cuándo se requiere?



Tolerancia a fallos

Reintentos automáticos sin generar duplicados ni inconsistencias.



Entornos distribuidos

Las redes fallan. El mismo mensaje puede llegar varias veces.



Procesamiento de mensajes

En colas o eventos (at-least-once delivery) puede haber reentregas.



Pagos y operaciones críticas

Evita cargos o transacciones duplicadas.



APIs seguras y confiables

Permite a los clientes reintentar sin temor a efectos no deseados.

4. Más ejemplos

Operación / Escenario	¿Es idempotente?	Por qué
GET /users/123	✓	Solo lee información.
PUT /users/123	✓	Deja el recurso en el mismo estado.
DELETE /users/123	✓	Si el recurso ya no existe, el resultado sigue siendo "no existe".
POST /orders	✗	Cada ejecución crea un nuevo recurso.
PATCH /users/123	⚠ Dependiente	Depende de cómo se implemente. Si siempre establece el mismo valor, puede ser idempotente.

5. Buenas prácticas para lograr idempotencia



Usar identificadores naturales o keys de idempotencia (p. ej. un ID de transacción).



Diseñar operaciones PUT/DELETE como idempotentes.



Validar existencia antes de crear.



Usar restricciones únicas en la base de datos.



Registrar y controlar reintentos.

6. Mensaje clave



La idempotencia es esencial para construir sistemas **resilientes**, confiables y preparados para la realidad de los entornos distribuidos, donde las fallas y los reintentos son la norma, no la excepción.



En resumen

La idempotencia permite reintentar sin preocuparse por efectos no deseados.



Más confiabilidad



Menos errores



Mejor experiencia de usuario

Funcionamiento del patrón Back pressure



El insight incómodo

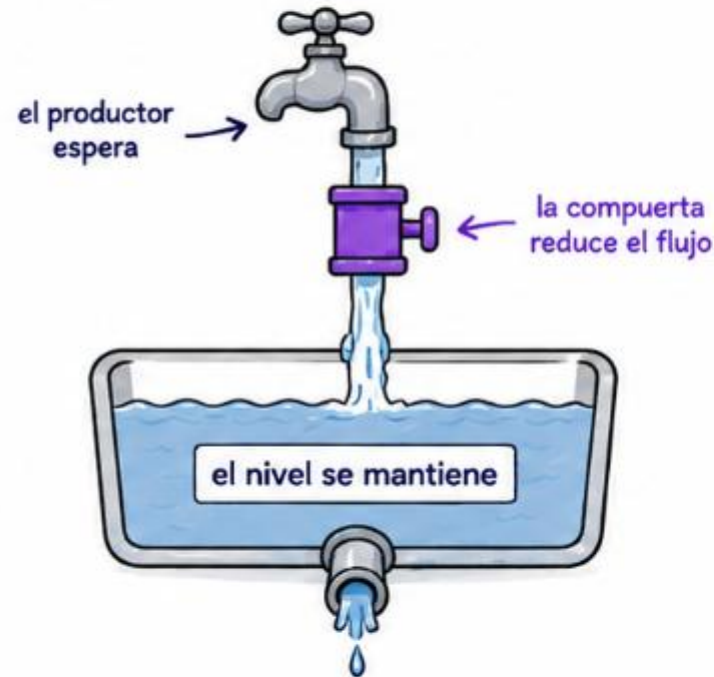
La presión hacia atrás frena a los productores antes de que las colas se desborden.

❌ SIN BACK PRESSURE



⚠️ la memoria se agota → crash

✅ CON BACK PRESSURE



Faucet = Productor
Genera datos o eventos.



Water = Cola (Queue)
Almacena temporalmente.



Drain = Consumidor
Procesa los datos.



Compuerta = Back Pressure
Reduce la velocidad cuando el sistema se satura.



TAKEAWAY

El throughput significa igualar la velocidad del productor con la **parte más lenta** del sistema.



Productor



Cola (Queue)



Consumidor

Funcionamiento del patrón Circuit Breaker

El interruptor de emergencia de las APIs



Protege tus sistemas.
Evita que una falla se convierta
en un problema mayor.

Sistemas
más resilientes
para un mundo
real

¿Qué es?

Es un patrón de diseño que evita fallas en cascada cuando un servicio externo está fallando o es lento. Actúa como un interruptor entre tu aplicación y un servicio externo. Si detecta fallas, "abre" el circuito y bloquea las peticiones por un tiempo.

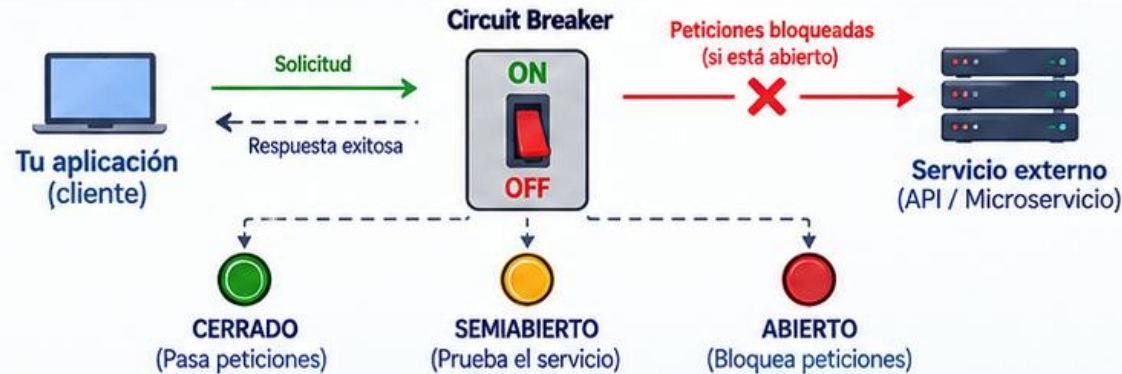


Resultado: sistemas más resilientes, estables y con alta disponibilidad.

¿Cómo funciona? (en 5 pasos)



Flujo de funcionamiento



¿Por qué usarlo?

- ✓ Evita fallas en cascada
- 👥 Protege servicios dependientes
- ⚙️ Mejora la resiliencia del sistema
- 🔄 Permite recuperación automática
- 📊 Aumenta la disponibilidad
- 👤 Mejor experiencia para el usuario



Actúa como un guardián entre tu aplicación y servicios inestables.

Estados del Circuit Breaker



Cerrado (Closed)

Todo funciona bien.
Las peticiones pasan.



Abierto (Open)

Se detectan fallas.
Las peticiones se bloquean.



Semiabierto (Half-Open)

Se permite una petición de prueba
para verificar si el servicio se recuperó.

Ejemplo práctico (llamada a un servicio de pagos)



Buenas prácticas

- ✓ Define umbrales de fallas adecuados
- ✓ Configura tiempos de espera razonables
- ✓ Registra y monitorea los estados
- ✓ Implementa fallback (respuesta alternativa)
- ✓ Úsalo junto con timeouts y reintentos
- ✓ Ajusta la configuración por servicio

Más control.
Más estabilidad.
Más confianza.

Aplicación de Domain-Driven Design (DDD)

Estación de Servicio

Gasolina · Aceite · Minimercado

Basado en el libro de Eric Evans — Domain-Driven Design

Universidad EAFIT - 2026

¿Qué es Domain-Driven Design?

El diseño guiado por el dominio del negocio

Filosofía Central

El software debe reflejar con precisión el dominio del negocio que modela.

La complejidad se domina poniendo el **MODELO** en el centro del diseño.

Fundamentos del Domain-Driven Design



Origen

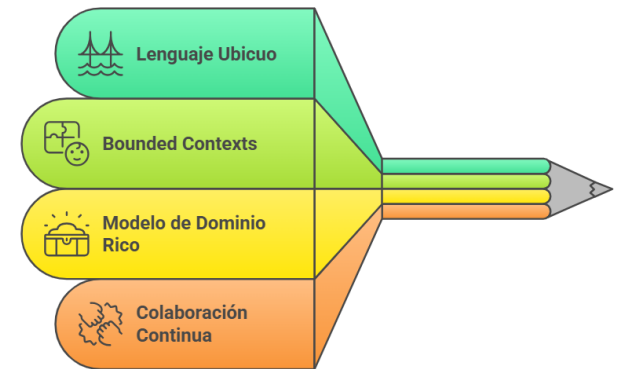
Eric Evans — 'Domain-Driven Design: Tackling Complexity in the Heart of Software' (2003).

El libro azul que cambió la arquitectura de software empresarial.

Pilares

1. Lenguaje Ubicuo (Ubiquitous Language)
2. Bounded Contexts
3. Modelo de Dominio Rico (Rich Domain)
4. Colaboración continua con expertos del negocio (Domain Experts)

Construyendo Software con DDD



Domain Driven Design (DDD)

De la realidad del negocio al software que genera valor



DDD conecta el mundo real del negocio con el diseño del software, utilizando un **lenguaje común**.

¿Qué es DDD?

Es un enfoque de diseño de software centrado en el dominio del negocio, que busca crear modelos compartidos entre expertos del negocio y del software para resolver problemas complejos.



Un mejor entendimiento del negocio, mejores soluciones de software.

Ejemplo: Hospital Infantil

Cuidando lo más importante: sus pacientes



¿Cómo funciona?

- Explorar el dominio**
Conversaciones entre expertos del negocio y el equipo de desarrollo (Event Storming, Workshops).
- Construir un Lenguaje Ubicuo**
Definir y usar los mismos términos (ej. Paciente, Cita, Diagnóstico, Tratamiento).
- Modelar el dominio**
Identificar entidades, value objects, agregados y sus relaciones.
- Diseñar e implementar**
El modelo del dominio guía la arquitectura y el código.
- Evolucionar constantemente**
Aprender del negocio y adaptar el modelo a nuevos requerimientos.

Elementos principales de DDD (con el ejemplo del hospital infantil)



Dominio

Área de negocio a modelar.

Ej.: Atención médica infantil



Lenguaje Ubicuo
(Ubiquitous Language)

Términos compartidos por todos.

Ej.: Paciente, Cita, Diagnóstico, Tratamiento



Entidad
(Entity)

Objeto con identidad única y ciclo de vida.

Ej.: Paciente
(Cada paciente es único)



Value Object

Describe un concepto del dominio, sin identidad propia.

Ej.: Información de contacto, Dirección, Rango de edad



Agregado
(Aggregate)

Conjunto de entidades y value objects que se tratan como una unidad.

Ej.: Historia Clínica
(Paciente + Citas + Diagnósticos + Tratamientos)



Repositorio
(Repository)

Abstracción para acceder y persistir agregados.

Ej.: Repositorio de pacientes



Dominio de Servicios
(Domain Service)

Lógica de negocio que no pertenece a una entidad en particular.

Ej.: Servicio de verificación de elegibilidad de seguro



Bounded Context

Límites del modelo de dominio y su lenguaje.

Ej.: Contexto de Atención Médica, Contexto de Facturación, Contexto de Farmacia

Beneficios



Mayor alineación negocio - tecnología



Software que resuelve problemas reales



Modelos más claros y mantenibles



Mayor capacidad de evolución



Equipos más colaborativos

“ Un modelo del dominio bien construido nos permite tomar mejores decisiones, hoy y en el futuro. ”

Resultado



Soluciones de software que reflejan el negocio, son más simples de mantener y generan mayor valor para los pacientes, sus familias y la organización.

Diagrama de Dominio con DDD

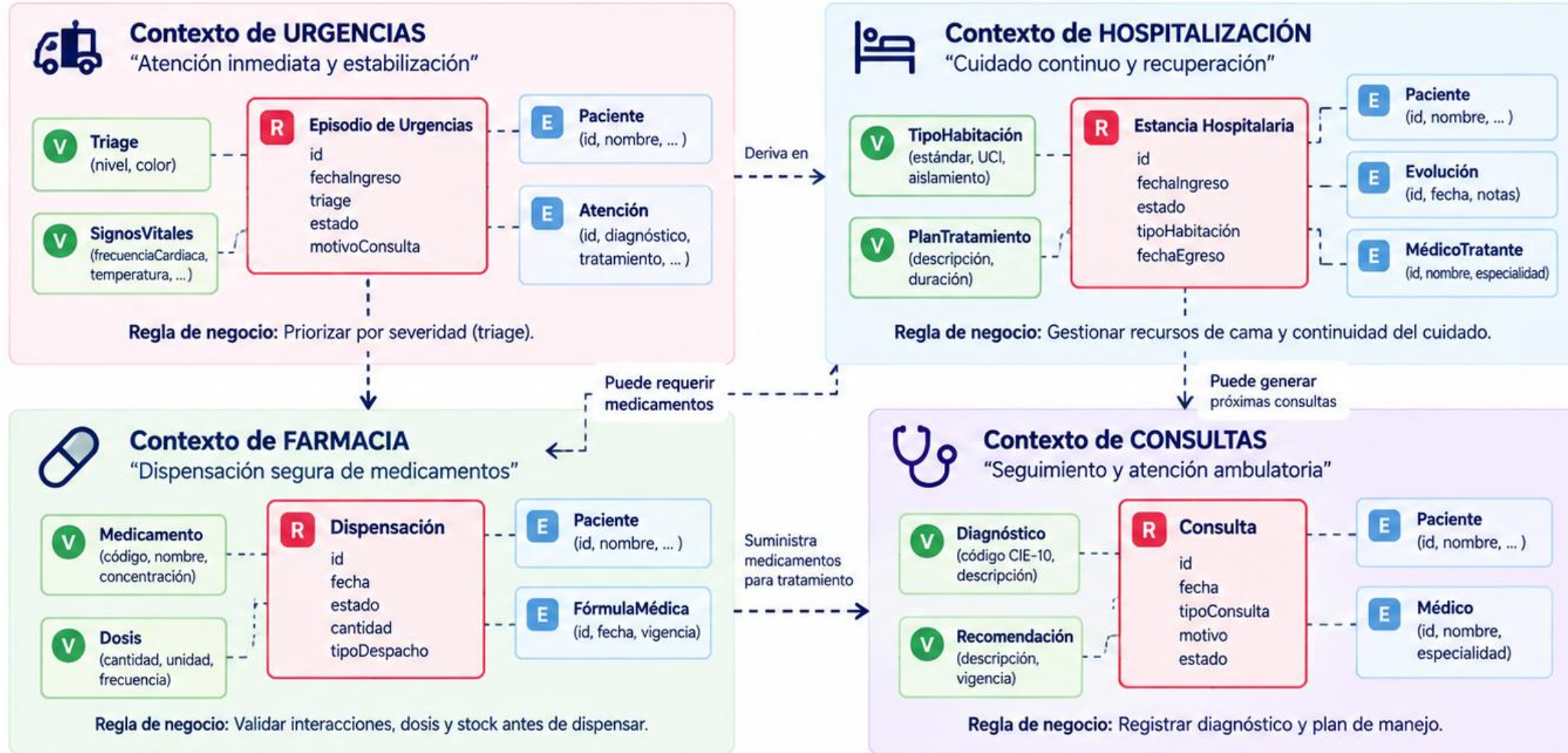
Ejemplo: Hospital Infantil



Un mismo dominio, múltiples contextos.
Cada contexto tiene su **propio modelo, lenguaje y reglas de negocio.**

Elementos de DDD

- R Aggregate Root**
Entidad principal del agregado.
Controla la consistencia.
- E Entidad**
Tiene identidad propia y ciclo de vida.
- V Value Object**
Sin identidad propia.
Describe un atributo del dominio.
- **Relación**
Asociación entre entidades / agregados.
- **Referencia entre Aggregates**
Solo por ID (evita acoplamiento).
- **Bounded Context**
Delimita el modelo, lenguaje y reglas.



Ejemplo de flujo del negocio

- 1 El paciente llega a Urgencias. →
- 2 Si requiere, se hospitaliza. →
- 3 Durante la estancia, se dispensan medicamentos desde Farmacia. →
- 4 Al egreso, se programan Consultas de seguimiento.



Valor de un modelo de dominio bien definido

- ✓ Lenguaje común entre negocio y tecnología
- ✓ Menor complejidad y mayor mantenibilidad
- ✓ Reglas de negocio claras por contexto
- ✓ Evolución independiente de cada módulo

Ejercicio en equipo: Modelar una estación de servicio

Aplicando Domain-Driven Design (DDD)



Escenario

Diseñar un sistema para una estación de servicio. Los requisitos incluyen gestionar el surtido de combustible, la venta de aceite, los pagos y el inventario de gasolina y diésel, así como la compra en un minimercado.



*Más que combustible,
movemos historias*



Tu reto

En equipo, diseñen el modelo de dominio de la estación de servicio utilizando DDD. **En 20 minutos** definan los siguientes elementos:

1



Lenguaje Ubicuo

Mínimo 10 términos con su definición.
Incluye: Surtidor: Máquina que despacha combustible.

2



Bounded Contexts

Identifiquen y justifiquen los contextos del dominio.

3



Entidades

Identifiquen las principales entidades y expliquen por qué lo son (tienen identidad).

4



Objetos de Valor

Identifiquen los objetos de valor y expliquen por qué no requieren identidad.

5



Agregados

Definan los agregados principales, su Aggregate Root y las reglas que protegen.

6



Reglas de negocio

Propongan al menos 8 reglas de negocio.

7



Caso de uso y caso de fallo

- Modelen el flujo: cliente compra combustible + minimercado + pago.
- Analicen el caso de fallo: inventario insuficiente de combustible.



Tiempo de trabajo

20 minutos

Organicen roles, discutan y lleguen a un consenso del equipo.



Trabajo en equipo



Escuchen diferentes puntos de vista.



Justifiquen sus decisiones.



Enfoquense en el dominio, no en la implementación.



Al final...

Prepárense para una breve socialización de sus resultados.

